

Exploring Interactive Composition Techniques with CsoundUnity and Unity

Xiaomeng (Susan) Zhong¹

¹ Berklee College of Music

² UC Santa Barbara

zhong0xm@gmail.com

Abstract. This paper presents different techniques and systems that were used to create an interactive composition using Csound, Unity and CsoundUnity. The paper discuss the creation of compositional and performative systems designed by combining the synthesis powers of Csound and the interactive game mechanisms in Unity. These systems includes: generative music with logic in C# played using Csound Instruments, trigger based control systems mimicking MIDI note on/off events using Unity's collision and rigidbody mechanics, transform object and controllers functioning as real-time controls like knobs and sliders. Taking advantage of both systems, it became possible to create a game-like composition *la forêt*.

Keywords: Csound, Cabbage, Unity, CsoundUnity, C#

1 Context

la forêt is an interactive music composition created for those needed a calm musical place to escape. It was made using Csound, Unity and CsoundUnity. Csound is the worlds most extensive audio programming language with unrivaled audio quality. Unity is a game engine that comes with a strong scripting API that contains complex mechanics that can be used simply, it also allows writing of custom C# scripts that determine game behaviors and mechanics. CsoundUnity is a wrapper that was written by Rory Walsh, and is based on Richard Henniger's Csound6Net for Unity, which is a cross-platform game engine. All these elements provide flexibility and endless possibilities, and allowed me to create *la forêt*, a CsoundUnity-based interactive composition that features generative harmony and melody, trigger based and continuous musical control system, as well as audio reactive environment.

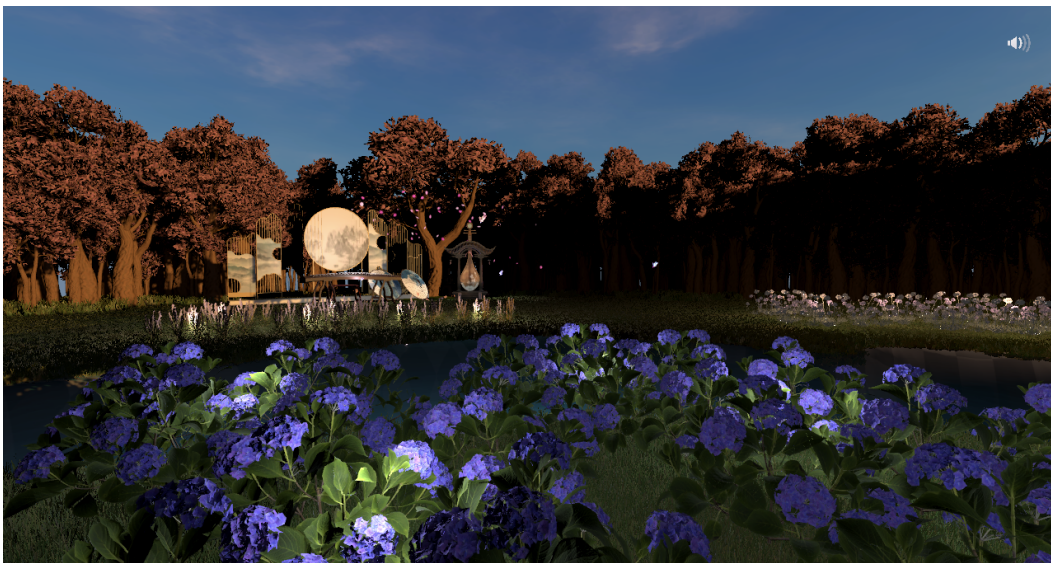


Fig. 1. *la forêt*

2 Generative Music System

In Csound, generative music is created using opcodes that generates new score events following a logic that determines the harmony, melody, and rhythm. Where rhythms are defined using metro, harmony and melody are defined using an array of possible notes. Following the same logic, the generative music system in *la forêt* was created in Unity using C# code, which allowed actions like key change, identifying key center and stating melodic sequence straightforward. A generative harmony system was created using the Markov chain, which is a stochastic model that decides a sequence of possible events based on probability of each. This was done by creating a 2D float array matrix that indicates the probability of the next chord progression for each tonality. This matrix was then developed with additional arrays and functions to create the generative harmonic system. As the harmony is determined, the key center for melody was chosen and passed on to the melody script, where, depending on the type of chord harmonically, MIDI values are added to the root note, to ensure the melody will always harmonize with the chord. The chosen harmony and melodic sequence were then stored as string variables. Different melodic sequences were built using logic and delay was used to achieve the goal easily. The tempo is controlled using a coroutine, which pause and resume these actions at specified time intervals. The action of picking chords, melody and anything additional will be done in this coroutine. All the logic was completed using C#. Csound then received a simple score event that uses the function: *SendScoreEvent(string scoreEvent)* in the format of “i\ “instrName\”startTime duration p4 p5 ...”.

```
string[] chords = { "imaj", "iimin", "ivmaj", "vmaj", "vimin" };

float[,] harmonyMatrix = new float[,]  
{  
    {0.0f, 0.15f, 0.4f, 0.3f, 0.15f},  
    {0.2f, 0.0f, 0.3f, 0.5f, 0.0f},  
    {0.4f, 0.0f, 0.0f, 0.4f, 0.2f},  
    {0.6f, 0.0f, 0.1f, 0.0f, 0.3f},  
    {0.1f, 0.4f, 0.3f, 0.2f, 0.0f}  
};
```

Fig. 2. Markov chain harmony matrix

3 Trigger-based System

In *la forêt*, most interactions with the environment were created based on the trigger systems. Unity has a built-in trigger and collision detection system that functions when a collider component and a rigidbody component is attached to the game object. Any objects that carry these two components can interact with other game objects that also carry those, or with controllers, giving a similar idea as pressing a key on a keyboard. The trigger message was then used to either send a score event or updating a k-rate variable through CsoundUnity’s *SetChannel(string channel, MYFLT val)* function. Just like in Cabbage, a k-rate variable is linked to a string channel via *chnget*, the string channel in Csound/Cabbage must share the exact same name as the one in Unity, the MYFLT value will then be passed through to the matching k-rate variable. In *la forêt*, each type of flower represents a different instrument and a different melodic sequence. As the player approaches the flower bed, a new melodic sequence will be triggered. When the player interact with the flower, events like playing an additional note, changing the melodic sequence, or changing the timbre will occur. The scene also features two traditional Chinese instruments, the Pipa and the Guzheng. Pressing on the string will trigger a note, and depending on the relative position being pressed, the pitch of the note varies.

```

private void OnTriggerEnter(Collider other)
{
    if (csoundUnity != null)
    {
        // Pick a random note from the melodyNotes array
        int randomNote = melodyNotes[Random.Range(0, melodyNotes.Length)];

        // Play the selected random note
        PlayNote(randomNote);
    }
    else
    {
        Debug.LogWarning("CsoundUnity reference not set in Note_1.");
    }
}

void PlayNote(int note)
{
    // Play the note using CsoundUnity
    string scoreEvent = string.Format("i\\\"SYNTH\\\" 0 2 {0}\", note);
    csoundUnity.SendScoreEvent(scoreEvent);
    Debug.Log("Playing note: " + note);
}

```

Fig. 3. Sending Csound score event using trigger system

4 Designing Real-time Control Tools

Real time control systems function in the same way as knobs and sliders. In Unity, there are many mechanics that can be designed for real-time controls, such as the relative position, rotation, and scale of a gameobject, and relative cursor position on screen. The position change and rotation angle of an object can be identified using the Transform component, a component that exists on all game objects. The mouse position can be coordinated using Unity's *Input.mousePosition*. Once the minimum and maximum of the game object is identified, it can then be scaled to an appropriate range that matches with the target k-rate variable range in Csound. These functions would be put within Unity's *void update()*, which are being called every frame and thus can vary depending on device's capabilities. New values are sent using *SetChannel(string channel, MYFLT val)* controlling musical parameters in real time, like reverb size, filter cutoffs, or amplitude and frequency, allowing a whole new level of musical interactivities.

```

//Receiving rotation informations
float pitch = transform.rotation.z; //0 to -0.3, pitch
float roll = transform.rotation.x; // -0.3 to 0.3, amplitude
float yaw = transform.rotation.y; // -0.7 to 0.7, filter cutoff

//Scaling the value using a different method
targetFreq = MapValue(pitch, 0f, -0.3f, 300f, 200f);
currentFreq = Mathf.Lerp(currentFreq, targetFreq, lerpSpeed * Time.deltaTime);
pan = Mathf.Min(Mathf.Max(MapValue(roll, 0.3f, -0.3f, 0f, 1f), 0f), 1f);
lpcutoff = (currentFreq + Mathf.Min(Mathf.Max(MapValue(yaw, -0.7f, 0.7f, 400f, 2000f), 400f), 2000f));

//Feeding into csound
csoundUnity.SetChannel("s_freq", currentFreq);
csoundUnity.SetChannel("s_pan", pan);
csoundUnity.SetChannel("s_lpcutoff", lpcutoff);

```

Fig. 4. Real-time continuous control using transform object.

```

// Update is called once per frame
void Update()
{
    TrackMouseInput();
    RealTimeCTRL();
}

void TrackMouseInput()
{
    //Get's mouse's relative position
    mouseX = Input.mousePosition.x / Screen.width;
    mouseY = Input.mousePosition.y / Screen.height;
}

void RealTimeCTRL()
{
    //Scale it to appropriate range, sends to csound
    float cutoff = Mathf.Lerp(10000, 100, Mathf.Clamp01(mouseX));
    float pan = Mathf.Lerp(0, 1, Mathf.Clamp01(mouseY));

    csoundUnity.SetChannel("cutoff", cutoff);
    csoundUnity.SetChannel("pan", pan);
}

```

Fig. 5. Real-time continuous control using mouse position

5 Controlling Game Parameter Using Audio

Part of creating an immersive environment involves creating an interactive audio system. In the piece, Csound was also used to create procedural generated ambiances, including wind and crickets. The procedural wind was used to control both audio and visual feedback in the scene. The procedural wind instrument was made in Csound, then the control parameters was done in C#. These control parameters includes amplitude, filter cutoff frequency and filter resonance. The values were then passed on to control game mechanics. The amplitude and cutoff frequency was used to control the speed and strength of the wind, that had an affect on the grass and leaves in the scene. The amplitude was also used to control the volume of the rustling leaves samples.

6 Beyond Sound

All these control systems create endless possibilities on the types of interactive compositions, allowing re-creation of real-life instruments in a digital space, or designing new ways to compose. With Unity being a game design platform, artists can create more than just a sonic environment. Visual elements can also be designed, adapting to real-time audio changes and a different interactivities can be created based on Unity's game mechanic systems, allowing artists to create a more harmonious interactive audiovisual experience. These techniques not only allowed the creation of *la forêt*, but have also been employed in a research project at MIT Nano Lab. Currently, they are being used to facilitate the recreation of the 'classic' Csound piece - *wiiSoundQuest* in VR that will feature a new form of interactive visual score whose gestures, contours, moving lines and shapes the players will trace, imitate, and follow.

7 The References Section

References

1. Boulanger, Richard. The Csound Book - Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming
2. Ferrone, Harrison. Learning C# by Developing Games with Unity 2021. 6th ed.
3. Cabbage Audio, cabbageaudio.com/
4. The Canonical Csound Reference Manual, csound.com/docs/manual/index.html.
5. CsoundUnity, rorywalsh.github.io/CsoundUnity/#/
6. Unity - Scripting API, docs.unity3d.com/ScriptReference/
7. Garden Instruments and Scripts, https://github.com/gwichanist0v0/Garden_ScriptInstrument.git